

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

Звіт
з Лабораторної роботи №3
з дисципліни: «Основи програмної інженерії»
на тему:
**«Рефакторинг коду та проведення Code Review
за індустріальними стандартами»**

Виконала: ст. гр. ПЗПІ-25-3
Гиренко В. О.

Прийняв: Гребенюк В. О.

Харків 2026

РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

МЕТА РОБОТИ

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

ХІД ВИКОНАННЯ РОБОТИ

1.1 Code Review (взаємний)

Таблиця 1.1 – Результати Code Review для Анастасії

№	Рядок коду	Проблема	Категорія	Рекомендація
1	Рядок 9	Зміна вмісту списку ValidStyles можлива ззовні системи через публічний доступ.	Mutable State / Порушення інкапсуляції	Змінити тип колекції на IReadOnlyList<string> або оголосити як масив (string[]).
2	Рядок 31	Однолітерний параметр s всередині лямбда-виразу Any() погіршує читабельність.	Poor Naming / Погане іменування	Перейменувати змінну s на більш зрозуміле і контекстне validStyle.
3	Рядки 44–53	Вкладена конструкція if-else всередині логіки обробки пасти/рецептів створює зайву складність.	Cognitive Complexity / Зайва складність	Спростити ієрархію умовних розгалужень за допомогою сучасного виразу switch.

4	Рядок 80	Перехоплюється лише <code>ArgumentException</code> . Метод <code>GenerateHook</code> може викинути інші типи винятків.	Specific Exception Handling / Слабкий перехоплювач	Замінити <code>ArgumentException</code> на базовий клас <code>Exception</code> , щоб уникнути раптового аварійного падіння програми.
5	Рядок 91	Публічний модифікатор <code>set</code> для властивості <code>Topic</code> дозволяє змінювати записи в історії після створення.	Data Integrity / Мутабельність моделі даних	Забезпечити незмінність (імутабельність) записів, замінивши модифікатор <code>set</code> на <code>init</code> .

Посилання на PR з коментарями Code Review - <https://github.com/anastasiadobroskokina/hookcraft/pull/1>

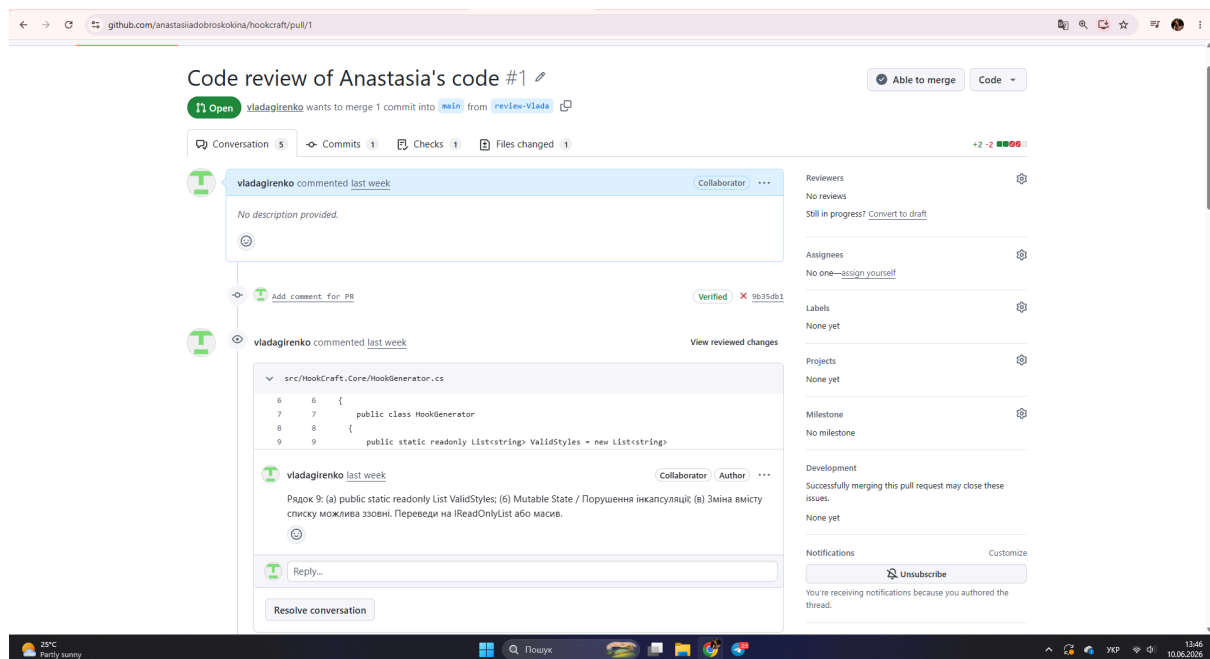


Рисунок 1.1 - Скріншот з вкладки Code Review

1.2 Результати рефакторингу власного коду

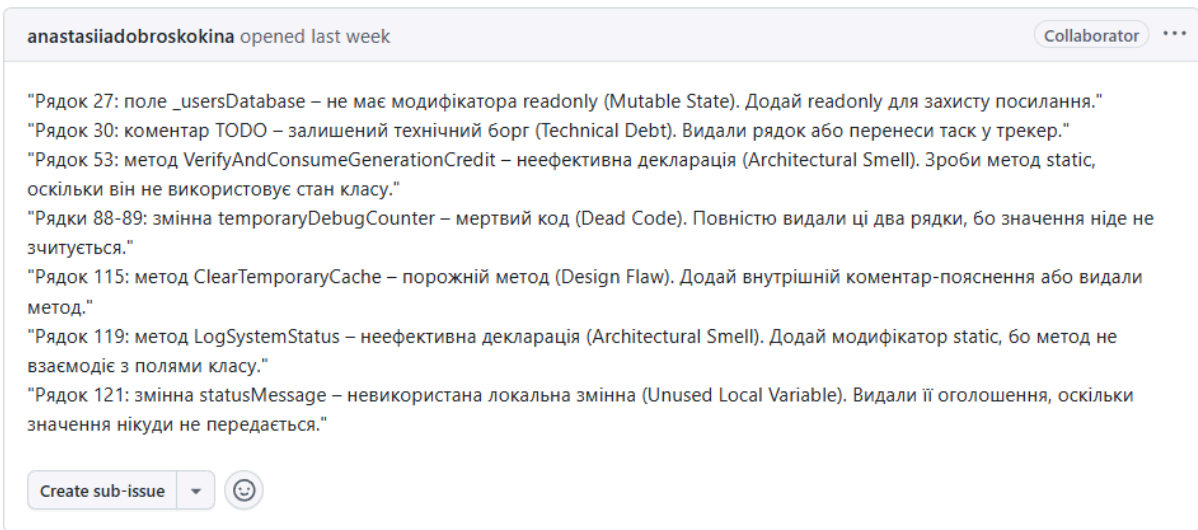


Рисунок 1.2 - Скріншот з Code Review від Анастасії

Таблиця 1.2 – Результати рефакторингу власного коду

№	БУЛО (фрагмент коду)	СТАЛО (фрагмент коду)	ЧОМУ (обґрунтування)
1	<code>private List<UserProfile> _usersDatabase = new List<UserProfile>();</code>	<code>private readonly List<UserProfile> _usersDatabase = new List<UserProfile>();</code>	Захист від мутації стану Додавання модифікатора <code>readonly</code> гарантує, що посилання на колекцію бази даних не буде випадково перезаписане або занулене з інших методів класу після ініціалізації.
2	<code>// TODO: Додати хешування паролів перед збереженням користувача</code>	(Рядок з коментарем повністю видалено з вихідного коду)	Усунення технічного боргу Тимчасові мітки <code>TODO</code> засмічують кодову базу. Усі завдання щодо майбутнього розширення функціоналу (хешування) мають зберігатися та відстежуватися у таск-трекері Jira, а не в коді.

3	<pre>int temporaryDebugCounter = 0; temporaryDebugCounter = hookId + 1;</pre>	(Фрагмент повністю видалено з методу ToggleFavorite)	Очищення від мертвого коду Ця змінна використовувалася лише для локального налагодження. Оскільки її значення надалі ніде не зчитується, вона створювала зайвий інженерний шум і погіршувала читабельність.
4	<pre>public void LogSystemStatus() { string statusMessage = "System is running smoothly"; }</pre>	<pre>public static void LogSystemStatus() { // Логування стану без взаємодії з об'єктом }</pre>	Усунення архітектурного запаху Метод не використовує стан екземпляра класу, тому має бути статичним. Невикористана локальна змінна statusMessage видалена для усунення попередження компілятора warning CS0219.
5	<pre>public void ClearTemporaryCache() { }</pre>	(Метод повністю видалено)	Усунення дефектів проєктування Порожні методи без реалізації та внутрішніх коментарів створюють хибне враження незавершеної логіки програми та збільшують когнітивне навантаження при читанні коду.

1.3 Звіт регресійного тестування

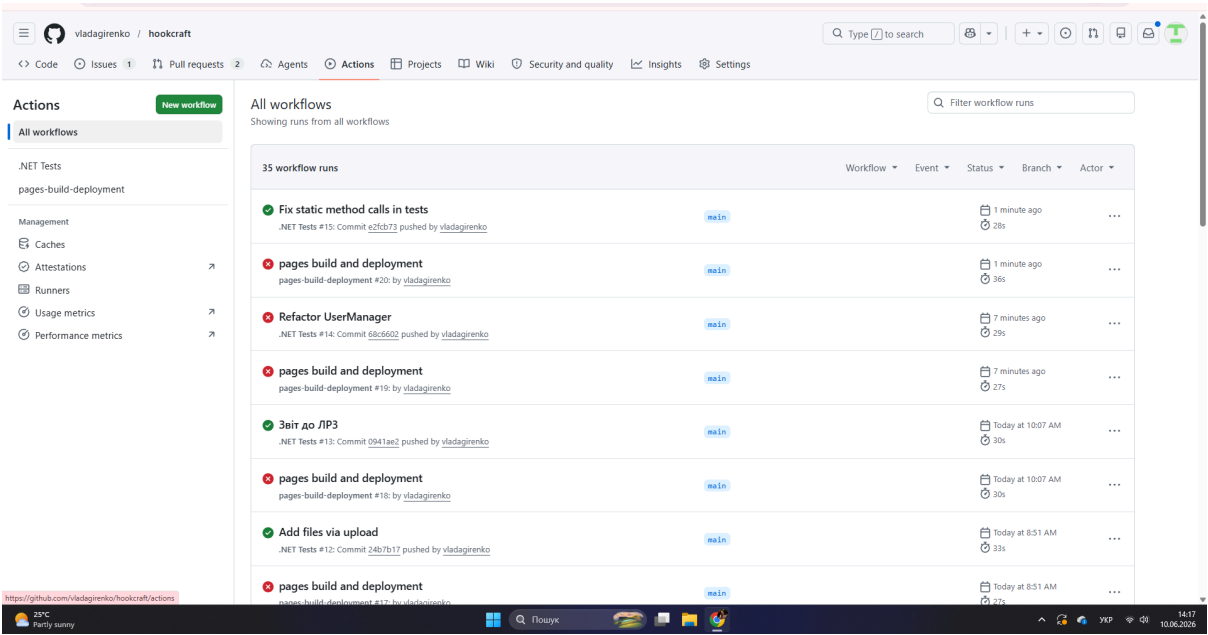


Рисунок 1.3 - Скріншот результату

1.4 Порівняння метрик «ДО / ПІСЛЯ»

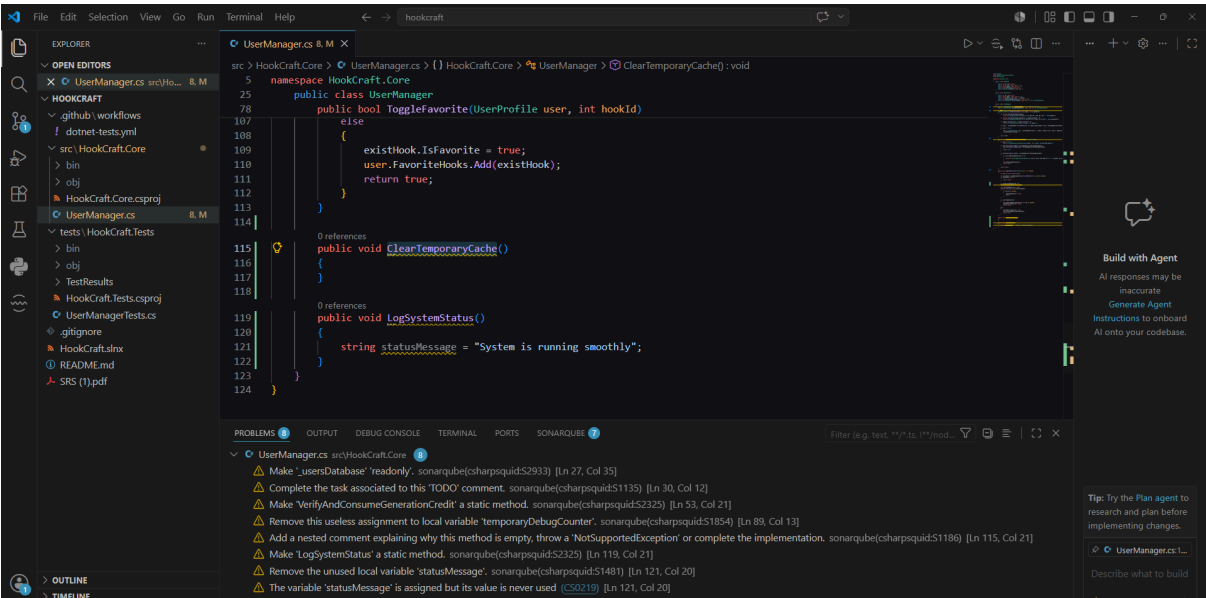


Рисунок 1.4 - Кількість SonarLint issues до (7)

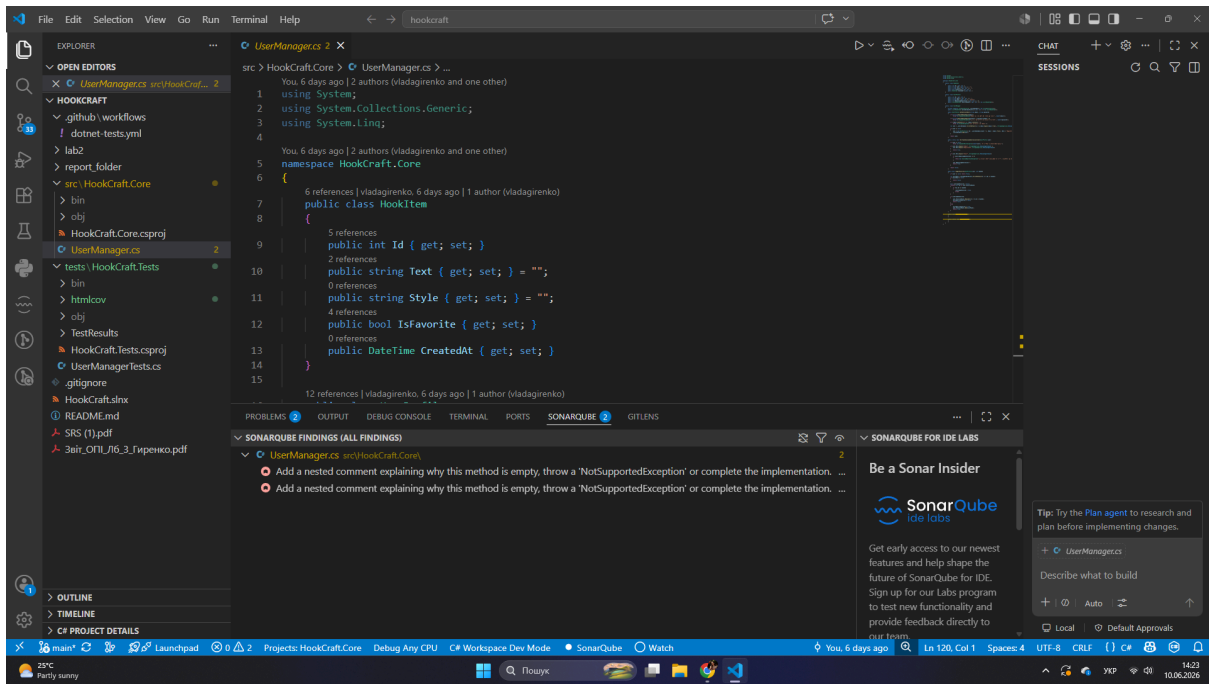


Рисунок 1.5 - Кількість SonarLint issues після (2)

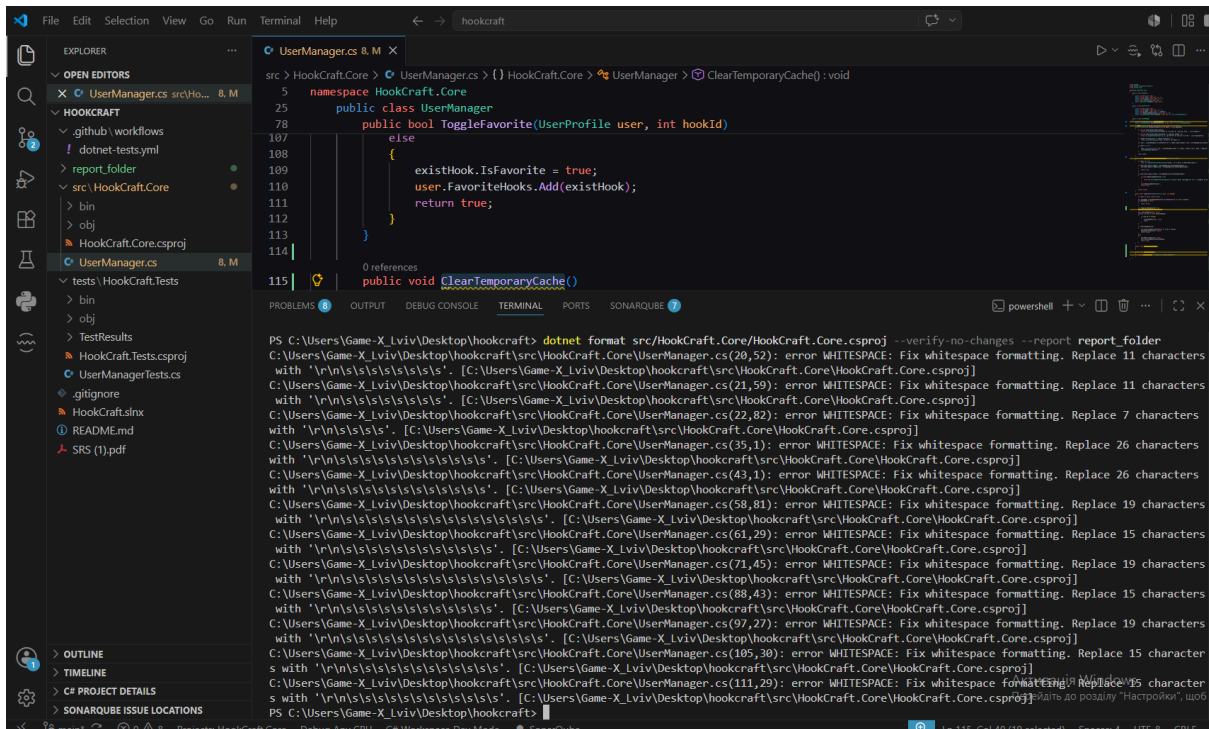


Рисунок 1.6 - Результат літера коду до

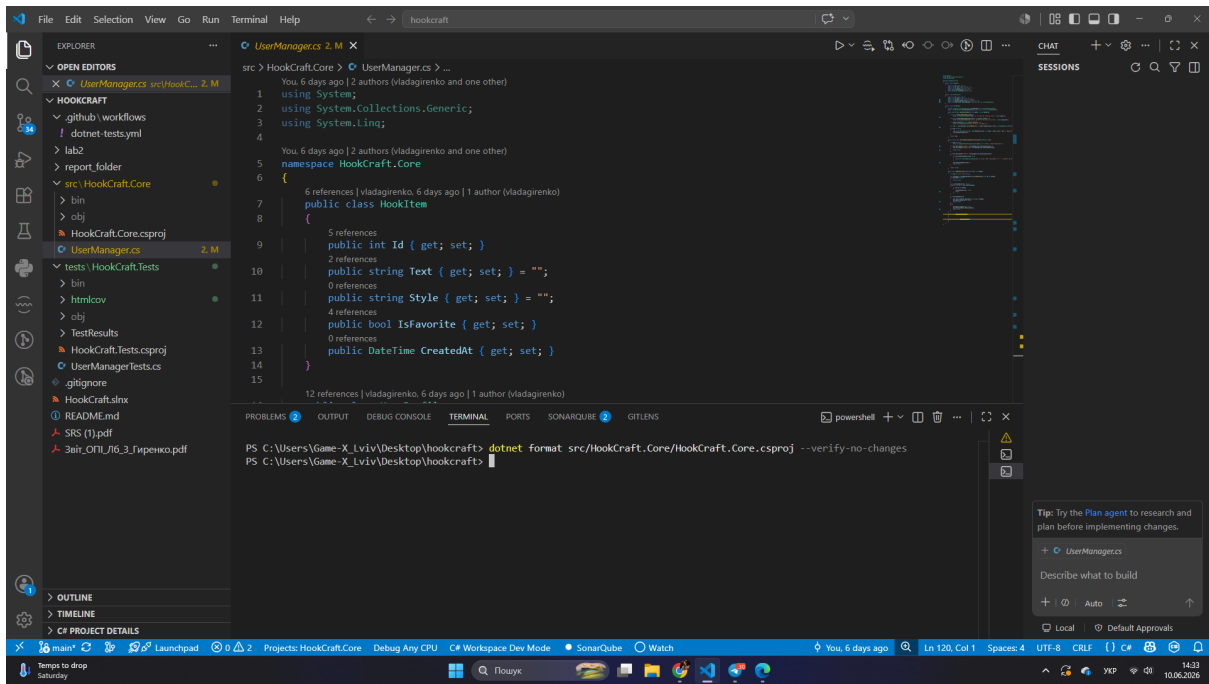


Рисунок 1.7 - Результат літера коду після

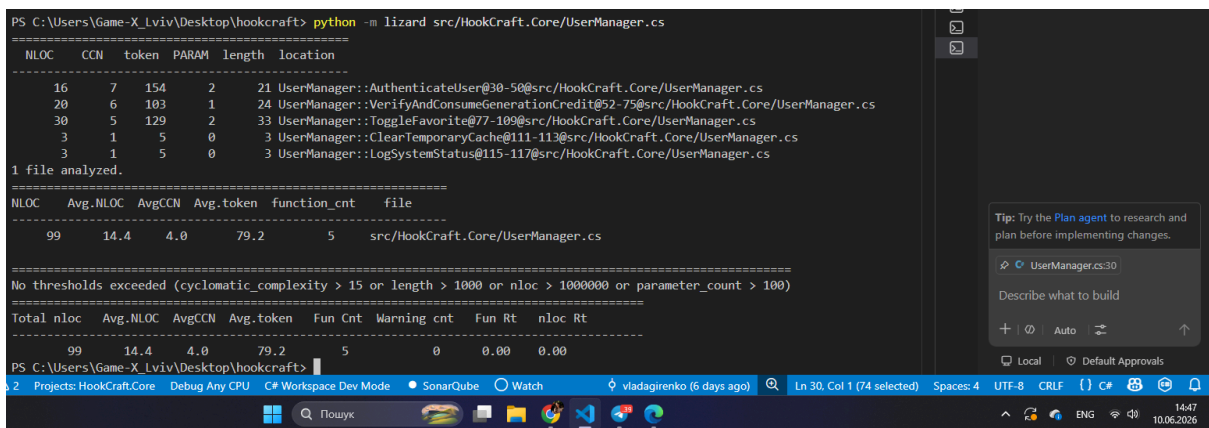


Рисунок 1.7 - Результат метрик складності після

1.5 Використання GitHub Copilot Chat для отримання пояснення фрагмента коду

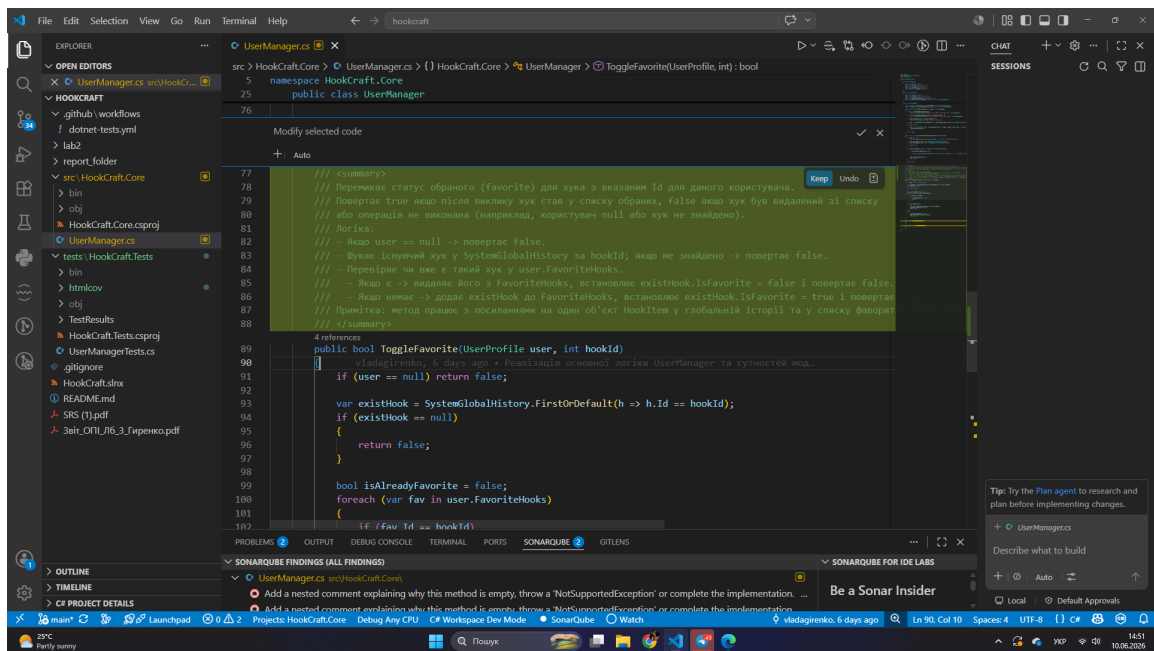


Рисунок 1.7 - Скріншот пояснень в Inline Chat

ВИСНОВКИ

Найціннішим досвідом під час спільного Code Review з Анастасією став практичний аналіз коду з позиції стороннього спостерігача, що дозволило чітко побачити приховані архітектурні дефекти. Ми навчилися не просто помічати помилки форматування, а глибоко оцінювати кодову базу за такими критеріями, як когнітивна складність, мутабельність моделей та порушення інкапсуляції. Отримані зауваження допомогли мені критично переосмислити власний код, трансформувати його відповідно до принципів Clean Code та успішно закрити технічний борг. Цей процес командної взаємодії довів, що Peer Review є незамінним етапом для підвищення загальної стабільності та якості програмного продукту перед релізом.

<https://github.com/vladagirenko/hookcraft>